

EMPLOYEE DEMO

Java Programming Assignment

Contents
1. Algorithm
2. UML Class Diagram
3. Java Source Code
4. Output Screen - VS Code Style

Date: 27/09/2026

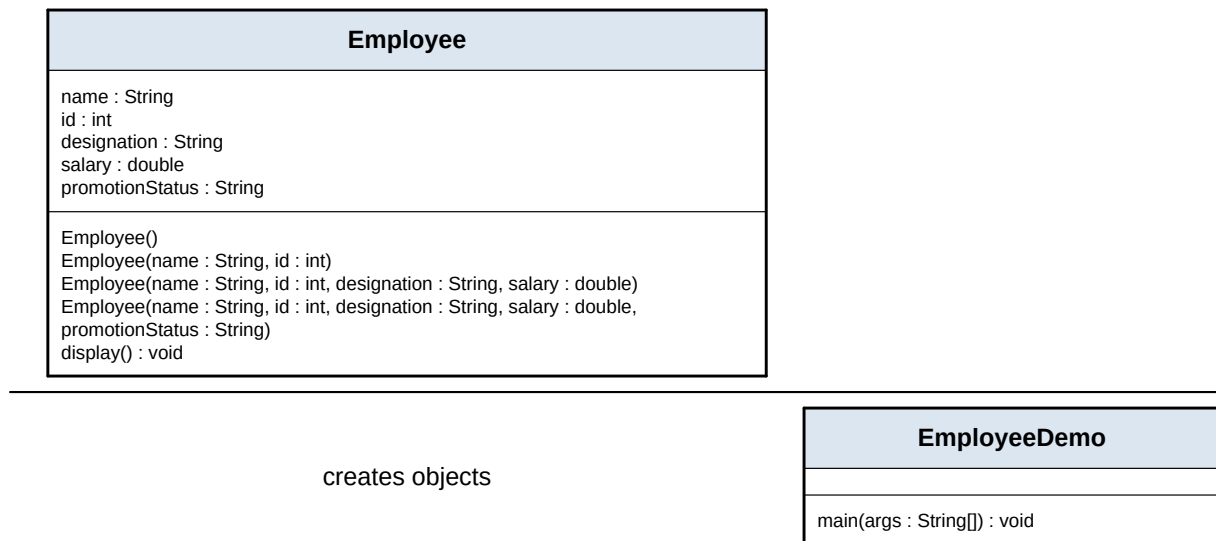
1. ALGORITHM

Steps for executing the EmployeeDemo program:

Step	Algorithm
1	Start the program.
2	Create an Employee object e1 using the default constructor.
3	Create Employee e2 using the constructor with name and ID.
4	Create Employee e3 using the constructor with name, ID, designation, and salary.
5	Create Employee e4 using the constructor with all five parameters.
6	Store the supplied values in the corresponding employee fields.
7	Call display() for e1, e2, e3, and e4.
8	Display each employee's name, ID, designation, salary, and promotion status.
9	Stop the program.

2. UML CLASS DIAGRAM

The Employee class demonstrates constructor overloading. EmployeeDemo creates objects using the different constructors.



UML: EmployeeDemo uses Employee and demonstrates overloaded constructors with 0, 2, 4, and 5 parameters.

3. JAVA SOURCE CODE

File name: **EmployeeDemo.java**

```
class Employee {
    String name;
    int id;
    String designation;
    double salary;
    String promotionStatus;

    // Default constructor
    Employee() {
        name = "Unknown";
        id = 0;
        designation = "Not Assigned";
        salary = 0.0;
        promotionStatus = "Not Decided";
    }

    // Constructor with 2 parameters
    Employee(String name, int id) {
        this.name = name;
        this.id = id;
        designation = "Employee";
        salary = 0.0;
        promotionStatus = "Pending";
    }

    // Constructor with 4 parameters
    Employee(String name, int id, String designation, double salary) {
        this.name = name;
        this.id = id;
        this.designation = designation;
        this.salary = salary;
        promotionStatus = "Pending";
    }

    // Constructor with all parameters
    Employee(String name, int id, String designation,
        double salary, String promotionStatus) {
        this.name = name;
        this.id = id;
        this.designation = designation;
        this.salary = salary;
        this.promotionStatus = promotionStatus;
    }

    void display() {
        System.out.println("Name           : " + name);
        System.out.println("ID           : " + id);
        System.out.println("Designation   : " + designation);
    }
}
```

3. JAVA SOURCE CODE (CONTINUED)

```
        System.out.println("Salary          : " + salary);
        System.out.println("Promotion Status : " + promotionStatus);
        System.out.println("-----");
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {

        // Using different overloaded constructors
        Employee e1 = new Employee();

        Employee e2 = new Employee("Ravi", 101);

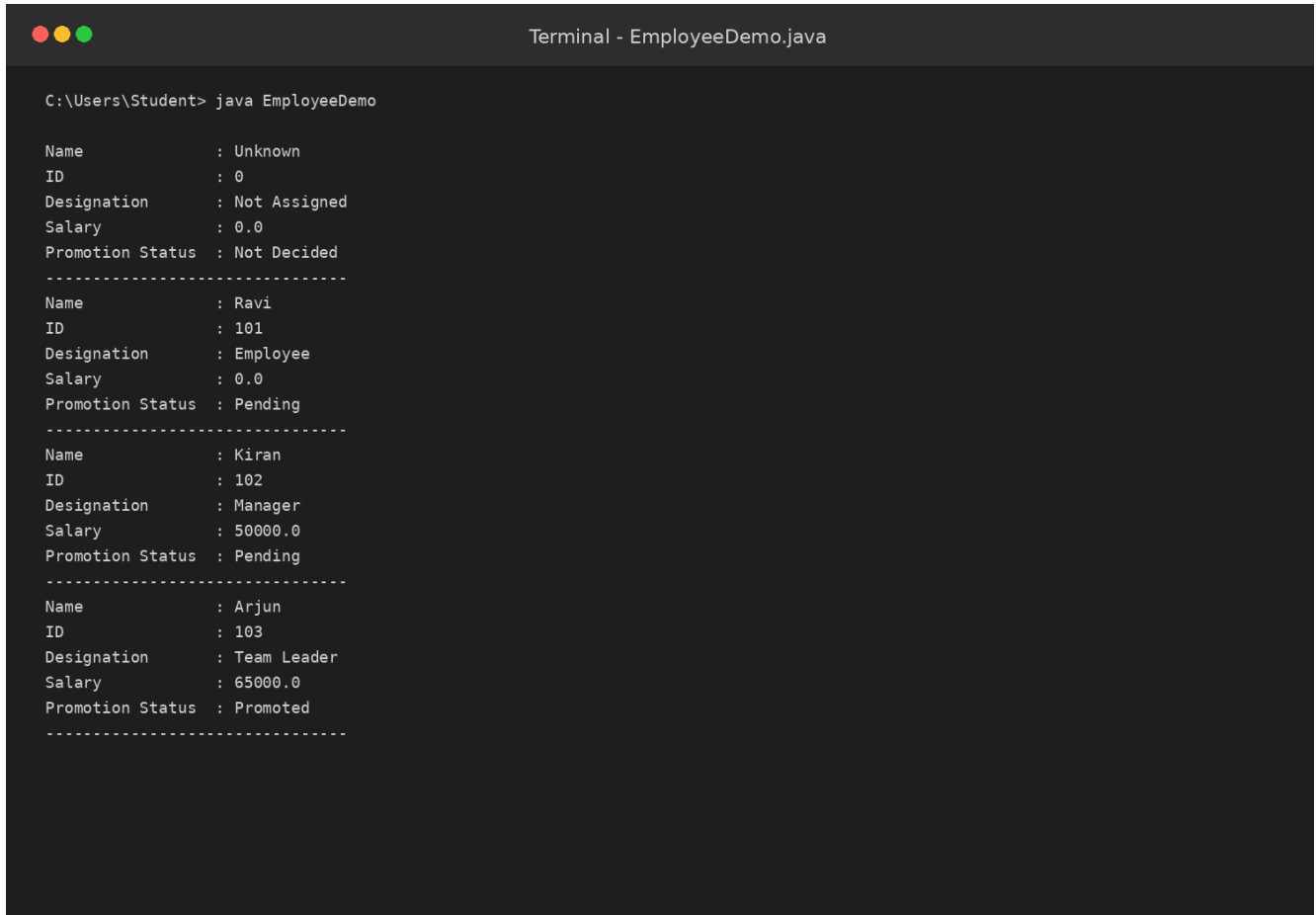
        Employee e3 = new Employee("Kiran", 102,
                                   "Manager", 50000);

        Employee e4 = new Employee("Arjun", 103,
                                   "Team Leader", 65000, "Promoted");

        // Display employee details
        e1.display();
        e2.display();
        e3.display();
        e4.display();
    }
}
```

4. OUTPUT SCREEN - VS CODE STYLE

Example execution of the program:



```
Terminal - EmployeeDemo.java

C:\Users\Student> java EmployeeDemo

Name      : Unknown
ID        : 0
Designation : Not Assigned
Salary    : 0.0
Promotion Status : Not Decided
-----
Name      : Ravi
ID        : 101
Designation : Employee
Salary    : 0.0
Promotion Status : Pending
-----
Name      : Kiran
ID        : 102
Designation : Manager
Salary    : 50000.0
Promotion Status : Pending
-----
Name      : Arjun
ID        : 103
Designation : Team Leader
Salary    : 65000.0
Promotion Status : Promoted
-----
```

Concept demonstrated: Constructor overloading using default, 2-parameter, 4-parameter, and all-parameter constructors.